

app.py

```

1 import streamlit as st
2 import pickle
3 import re
4 from sklearn.feature_extraction.text import TfidfVectorizer
5 from nltk.corpus import stopwords
6 import nltk
7 from ntscraper import Nitter
8
9 # Download stopwords once, using Streamlit's caching
10 @st.cache_resource
11 def load_stopwords():
12     nltk.download('stopwords')
13     return stopwords.words('english')
14
15 # Load model and vectorizer once
16 @st.cache_resource
17 def load_model_and_vectorizer():
18     with open('model.pkl', 'rb') as model_file:
19         model = pickle.load(model_file)
20     with open('vectorizer.pkl', 'rb') as vectorizer_file:
21         vectorizer = pickle.load(vectorizer_file)
22     return model, vectorizer
23
24 # Define sentiment prediction function
25 def predict_sentiment(text, model, vectorizer, stop_words):
26     # Preprocess text
27     text = re.sub('[^a-zA-Z]', ' ', text)
28     text = text.lower()
29     text = text.split()
30     text = [word for word in text if word not in stop_words]
31     text = ' '.join(text)
32     text = [text]
33     text = vectorizer.transform(text)
34
35     # Predict sentiment
36     sentiment = model.predict(text)
37     return "Negative" if sentiment == 0 else "Positive"
38
39 # Initialize Nitter scraper
40 @st.cache_resource
41 def initialize_scraper():
42     return Nitter(log_level=1)
43
44 # Function to create a colored card
45 def create_card(tweet_text, sentiment):
46     color = "green" if sentiment == "Positive" else "red"
47     card_html = f"""
48     <div style="background-color: {color}; padding: 10px; border-radius: 5px; margin: 10px
49     0;">
50         <h5 style="color: white;">{sentiment} Sentiment</h5>
51         <p style="color: white;">{tweet_text}</p>
52     </div>

```

```
52     """
53     return card_html
54
55 # Main app logic
56 def main():
57     st.title("Twitter Sentiment Analysis")
58
59     # Load stopwords, model, vectorizer, and scraper only once
60     stop_words = load_stopwords()
61     model, vectorizer = load_model_and_vectorizer()
62     scraper = initialize_scraper()
63
64     # User input: either text input or Twitter username
65     option = st.selectbox("Choose an option", ["Input text", "Get tweets from user"])
66
67     if option == "Input text":
68         text_input = st.text_area("Enter text to analyze sentiment")
69         if st.button("Analyze"):
70             sentiment = predict_sentiment(text_input, model, vectorizer, stop_words)
71             st.write(f"Sentiment: {sentiment}")
72
73     elif option == "Get tweets from user":
74         username = st.text_input("Enter Twitter username")
75         if st.button("Fetch Tweets"):
76             tweets_data = scraper.get_tweets(username, mode='user', number=5)
77             if 'tweets' in tweets_data: # Check if the 'tweets' key exists
78                 for tweet in tweets_data['tweets']:
79                     tweet_text = tweet['text'] # Access the text of the tweet
80                     sentiment = predict_sentiment(tweet_text, model, vectorizer,
stop_words) # Predict sentiment of the tweet text
81
82                     # Create and display the colored card for the tweet
83                     card_html = create_card(tweet_text, sentiment)
84                     st.markdown(card_html, unsafe_allow_html=True)
85             else:
86                 st.write("No tweets found or an error occurred.")
87
88 if __name__ == "__main__":
89     main()
```